Financial Sentiment using NLP

Assignment 1

General Guidelines

- We strongly discourage using LLMs for coding purposes at this stage. However, you are encouraged to leverage LLMs to understand concepts and explore newer ideas that you may not be aware of.
- This assignment contains two main tasks. After that, you will find a bonus questions section, which consists of extensions and improvements to the main tasks. These are completely optional, but they do carry brownie points ;)
- We highly encourage you to attempt the bonus questions. Our motive is to get you thinking about creative solutions to the problems mentioned in this section, while still maintaining accuracy. If you have an idea and would like to discuss it with us, feel free to do so.
- Towards the end, you will find an Appendix / Resources section containing materials to help you with the tasks. We strongly recommend going through the official library documentation provided there, as this is a good habit to develop going forward.
- We expect clean, structured, and modular code. The workflow should be organized into well-defined functions so that experiments are reproducible and results can be easily verified.

Question 1: Rule-Based Sentiment Analysis (Financial + General Lexicons)

In this task, you will build and evaluate rule-based sentiment analysis systems using:

- A financial-domain lexicon: **Loughran–McDonald sentiment dictionary**
- A general-purpose rule-based tool: **either TextBlob or VADER** (choose any one)

A. Text Preprocessing

Preprocess each sentence in the dataset using the following steps in the given order:

1. **Text Cleaning (using `re` library)**
    - Remove punctuation, digits, and special characters
    - Replace multiple spaces with a single space
    - Convert all text to lowercase
2. **Tokenization**
    - Split each cleaned sentence into individual tokens (words)
3. **Stopword Removal**
    - Remove common English stopwords using NLTK (or an equivalent stopword list)
4. **Stemming or Lemmatization**
    - Choose either stemming or lemmatization
    - Apply it uniformly to all tokens
    - Clearly mention which method you used and justify your choice

Include at least one example showing the transformation from raw text to final preprocessed tokens.

B. Sentiment Analysis – Part 1: Loughran–McDonald Financial Lexicon

Use the Loughran–McDonald (LM) sentiment lexicon provided in the Appendix.

Although the lexicon contains multiple sentiment categories, for this task, you must use only the following two columns:

- Positive
- Negative

Steps:

1. Extract two word collections:
    - A list (or equivalent structure) of positive words
    - A list (or equivalent structure) of negative words
2. For each preprocessed sentence:

- Count the number of words appearing in the positive list
- Count the number of words appearing in the negative list

3. Assign a sentiment label using the rule:
   - If positive_count > negative_count → Positive
   - If negative_count > positive_count → Negative
   - If both counts are equal, you may assign Neutral (optional)
4. Store the predicted sentiment label for each sentence.

C. Optimization of the LM-Based Rule System

The naive implementation of the LM-based method may be slow, as it requires checking each token against large word lists.

You are encouraged to optimize your implementation so that sentiment computation takes less time.

In your report, briefly explain what optimization you applied (if any).

D. Sentiment Analysis – Part 2: Choose ONE Method

In addition to the LM-based model, implement one of the following general-purpose rule-based sentiment methods.

Option 1: TextBlob

- Obtain the polarity score (range: -1 to $+1$) for each sentence
- Convert the polarity score into sentiment labels:
  - polarity $> 0 \rightarrow$ Positive
  - polarity $< 0 \rightarrow$ Negative
  - polarity $= 0 \rightarrow$ Neutral (optional)

Option 2: VADER (Valence Aware Dictionary)

- Obtain the compound score (range: -1 to $+1$)
- Convert the compound score using standard thresholds:
  - compound $\geq 0.05 \rightarrow$ Positive
  - compound $\leq -0.05 \rightarrow$ Negative
  - Otherwise $\rightarrow$ Neutral

Clearly mention which method you chose.

E. Evaluation Metrics

Evaluate both sentiment analysis methods:

- Loughran–McDonald rule-based method
- Your chosen TextBlob or VADER method

Compute and report:

$$\text{Correct Positive Percentage} = \frac{\text{Correctly predicted positive sentences}}{\text{Total positive sentences}} \times 100$$

$$\text{Correct Negative Percentage} = \frac{\text{Correctly predicted negative sentences}}{\text{Total negative sentences}} \times 100$$

$$\text{Overall Accuracy Percentage} = \frac{\text{Total correct predictions}}{\text{Total number of sentences}} \times 100$$

F. Deliverables

You must submit two files.

1. Python Notebook (.ipynb)

The notebook must include:

- Full preprocessing pipeline with explanations
- LM-based sentiment classifier implementation
- Optimized LM implementation and brief explanation
- TextBlob or VADER implementation
- A comparison table of true vs predicted labels
- At least two visualizations
- Clear comments and markdown explanations

2. Report (PDF)

The report must include:

- Overview of the task and dataset
- Explanation of preprocessing choices
- Description of both sentiment methods
- Explanation of optimization

- Accuracy metrics
- Visualizations
- Answers to conceptual questions

Question 2: Machine Learning–Based Sentiment Analysis

You will implement ML-based sentiment analysis models and compare them with rule-based approaches.

The dataset contains three sentiment classes:

- +1: Positive
- 0: Neutral
- −1: Negative

Part A: Feature Engineering

Apply appropriate preprocessing and convert text into numerical features using Bag-of-Words or TF–IDF. Mention all choices and parameters used.

Part B: Naive Bayes Classifier

Train and evaluate a Naive Bayes classifier. Clearly explain the variant used and report class-wise and overall metrics.

Part C: Logistic Regression Classifier

Train and evaluate a Logistic Regression classifier. Compare its performance with Naive Bayes.

Bonus Questions (Optional – Brownie Points)

Bonus questions extend the main tasks. Clearly state which bonus questions you attempt in the report.

Bonus Questions – Question 1

Bonus 1: Using Additional Lexicon Categories

The Loughran–McDonald lexicon contains several additional sentiment-related categories such as uncertainty, litigious, constraining, and strong/weak modality, beyond positive and negative. Try to use these categories to improve your model where it is currently failing. For example, you may generate separate confidence scores using these different categories.

Bonus 2: Handling Negation and Sarcasm

Rule-based sentiment analysis often performs poorly in the presence of **negation** (e.g., "not good") and **sarcasm**.

Propose and implement a heuristic-based improvement to handle:

- Negation (e.g., using n-grams as discussed in the meet)
- **OR** sarcasm

Bonus Questions – Question 2

Bonus 1: Implementing a Classifier from Scratch

Implement Naive Bayes or Logistic Regression from scratch and compare with library-based results.

Bonus 2: Handling Class Imbalance

Real-world sentiment datasets often exhibit class imbalance. Analyze the class distribution in the dataset. Apply any one imbalance-handling technique, such as:

- Class weighting
- Oversampling
- Undersampling

Bonus 3: Probability-Based Sentiment Scoring

Use Logistic Regression probabilities to define a continuous sentiment score.

Appendix / Resources

- **Python Regular Expressions (`re` library):** `re` — Regular expression operations (Python Documentation)
- **TextBlob Documentation:** TextBlob: Simplified Text Processing
- **VADER Sentiment Analysis:** VaderSentiment Documentation
- **NLTK Documentation:** Natural Language Toolkit (NLTK)

- **Logistic Regression (scikit-learn):** LogisticRegression — scikit-learn Documentation
- **Naive Bayes (scikit-learn):** Naive Bayes — scikit-learn Documentation
- **Datasets:** Google Drive Folder